開　南　大　學

資訊學系

碩士論文

網頁設計乙級技術士術科考試資安弱點防禦實作研究

指導教授：許榮隆 博士
研 究 生：　翁偉倬　撰

中 華 民 國 壹 佰 壹 拾 伍 年 陸 月

摘　要

隨著 Web 技術的快速發展，PHP 仍是當前建置動態網站的主流語言之一。然而，開發者若缺乏資安意識，容易在程式碼中留下安全弱點，導致使用資料外洩或系統遭駭。本研究旨在探討 PHP 網站開發過程中的安全性強化流程，透過實作、檢測、分析與修復的流程，提升 Web 應用程式的防禦能力。

本研究透過四階段的實作來說明並修正網頁設計乙級技術士術科考試四個考題的資安弱點

階段一: 在實驗環境完成網頁設計乙級技術士術科考試所要求的四個網站

階段二: 在實驗環境使用 OWASP ZAPV,分別對第一階端完成的四個網站作 Automated Scan,找出其資安的弱點,並參考文獻分別對不同種類的資安弱點做出說明

階段三: 更改網站程式以修復資安弱點

階段四: 重複階段二和三直至 Automated Scan,找不出資安的弱點

關鍵詞：網頁設計乙級技術士、PHP 網站資安、OWASP ZAP、網站弱點掃描

Abstract

目　錄

第一章　緒論

第一節　研究背景 ---------- 1

第二節　研究動機 ---------- 2

第三節　研究目的 ---------- 3

第四節　研究問題 ---------- 4

第五節　名詞解釋 ---------- 5

第二章 文獻探討

第一節　PHP 網頁設計乙級術科試題常見之弱點---------- 6

第二節　Web 網站安全評估工具---------- 10

第三節　PHP 網站因資安事件造成的損失---------- 15

第一章　緒論

第一節　研究背景

根據數位發展部112年核定的國家資通安全研究院發展目標及計劃[1]的緣起內容如圖(3-1),重視資安即是重視非政府機關的資訊資產,也是重視政府機關的資訊資產,更是重視國家的安全,透過相關法令的制定,如下圖(3-2),使得相關的產業得以全面發展

壹、總目標及說明

一、緣起

我國政經情勢特殊，除面對全球複雜多元之資通訊變革，尚需面對較其他國家更為險峻之資安威脅，為確保民眾數位生活福祉、新興資安產業發展及數位國土國家安全，持續落實並精進各項資通安全防護工作實屬必要。在「資安即國安」政策下，政府已將資通安全提升至國家安全層級，亦確立邁向數位國家之具體方向。

圖 1-1-1

圖 1-1-2

第二節　研究動機

網絡安全是當前資訊安全領域的核心課題，而確保網站健全的關鍵，在於開發者是否具備完善的安全開發能力（Secure Coding）。然而，現行網頁設計師的培訓體系在資安教育方面仍顯不足。以國家「網頁設計乙級技術士」職訓課程為例，如圖(1-2-1) 所示，課程內容多聚焦於功能實現，鮮少觸及如何編寫具備防禦性的程式碼以避免安全漏洞。

此現象主因在於術科評核標準係以「功能達成」為唯一導向，導致開發過程往往忽略資安要求。基於此，本研究之動機在於透過實際開發「網頁設計乙級技術士」術科考試之四項題組案例，深入探討如何針對標準化考題進行資安弱點測試，並提出相應的修復與強化方案，藉此彌補現行技術認證與實務資安需求間的斷層。

課程內容

大綱:
1. 一般與共同課程(53小時)：報到開訓、輔導管理、生活津貼、職業訓練與就業服務資源介紹、職業安全衛生等
2. 資料庫程式設計(160小時)：XAMPP安裝設定、PHP程式語言、MySQL資料庫操作、SQL語法、資料庫管理概念、Git版本控制等
3. 網頁動態技術(160小時)：javascript、jQuery、Bootstrap、RWD響應式網頁製作、Ajax技術應用等
4. 網頁排版編輯(80小時)：HTML5、CSS3、網頁編輯器及工具使用等
5. 視覺影像設計(120小時)：Photoshop數位影像處理、Illustrator向量繪圖設計等
6. 數位媒體應用(115小時)：API串接、RESTful API、、MVC開發模式介紹、Laravel framework操作、網頁實作技術整合應用等
7. 資訊網路概論(32小時)：資訊架構及網路應用概論等
8. 網頁設計實務(200小時)：網頁設計乙級檢定輔導:「卓越科技大學校園資訊系統」網站、「健康促進網社群平台」網站、「ABC影城」網站、「精品電子商務」網站

圖 1-2-1

第三節　研究目的

根據研究背景與動機，本研究欲達成之研究目在建立一個完整的 Web 資安強化流程，具體目標如下：

一、系統實作：根據技術士技能檢定網頁設計乙級術科測試試題(附錄1)使

用 PHP 語言開發4個網站

二、漏洞檢測：利用 OWASP ZAP 對目標網站進行全方位的自動化掃描，識別潛在的安全漏洞。

三、風險分析：根據掃描結果的報告，分析漏洞的成因及其可能造成的損害等級。

四、程式修復：針對偵測出的漏洞，提出對應的程式碼修正方案並進行覆測，確保漏洞已被有效排除。

第四節 研究問題

本研究擬探討並回答以下關鍵問題：

一、在未經資安強化的 PHP 網站中，最常見的安全漏洞類型為何？

二、OWASP ZAP 掃描工具對於偵測不同類型的 PHP 網站漏洞（如注入攻擊、認證失效等）之有效性為何？

三、如何透過修改 PHP 程式碼來根除掃描工具所發現的安全威脅？

四、修復漏洞後的系統在功能面與效能面是否會受到顯著影響？

第五節 名詞解釋

為了便於理解本研究內容，針對相關術語定義如下表 1-5-1

名詞 (Term)	定義與解釋 (Definition & Explanation)
PHP	全稱為 Hypertext Preprocessor，是一種伺服器端（Server-side）開源腳本語言。其特性為可嵌入 HTML 中執行，在本研究中作為開發「網頁設計乙級」術科系統之核心後端語言。
OWASP ZAP	全稱為 Zed Attack Proxy，由開放網路軟體安全計畫（OWASP）維護之開源動態掃描工具。本研究利用其主動與被動掃描功能，對 PHP 系統進行自動化弱點辨識。
SQL 注入 (SQL Injection)	一種代碼注入攻擊，係指攻擊者利用應用程式對輸入過濾的疏漏，將惡意 SQL 指令注入後端資料庫查詢語句中，藉此達成非法讀取、竄改或刪除資料庫資料之目的。
跨站腳本 (XSS)	全稱為 Cross-Site Scripting，係指攻擊者在網頁中嵌入惡意用戶端腳本（如 JavaScript）。當其他用戶瀏覽該頁面時，腳本將在用戶瀏覽器執行，可能導致會話劫持（Session Hijacking）或個資外洩。
DAST	全稱為 Dynamic Application Security Testing（動態應用程式安全測試）。係指在系統執行狀態下，透過黑箱測試（Black-box Testing）模擬外部攻擊行為，以檢測網站是否存在安全漏洞的測試方法。
Secure Coding	安全開發規範。指在程式開發過程中，遵循特定的安全性標準（如輸入驗證、參數化查詢），從撰寫階段即預防漏洞產生，是本研究提出之修復策略的核心依據。
CVSS	全稱為 Common Vulnerability Scoring System（通用漏洞評分系統）。為一套標準化的漏洞風險評估框架，本研究依據其分值對掃描結果進行量化評級，以決定修復優先順序。
OWASP Top 10	由開放網路軟體安全計畫發布的十大 Web 應用程式安全風險清單。本研究以此作為評估「網頁設計乙級」術科試題安全程度的主要國際標準參考。
Regression Testing	回歸測試。指在漏洞修復程式碼提交後，重新執行測試以驗證修復效果，並確認安全強化過程未對原有的術科功能（如資料增刪查改）造成負面影響。
Baseline	基準線。在本研究中定義為「完全依照勞動部勞動力發展署提供之術科參考範例所建置之原始系統」，作為後續資安強化與對比分析的對照組。

表 1-5-1

第二章 文獻探討

第一節　PHP 網站安全

PHP 8.5 作為 PHP 8 系列的最新演進，不僅在執行效能上有所突破，更深刻體現了「預設安全（Secure by Default）」的設計哲學。隨著網頁威脅模式的轉變，PHP 8.5 針對傳統弱點（如型別混淆、記憶體溢位、不當反序列化）引入了更為強大的語言級防護機制。

一、 語言核心的安全架構強化

(一)PHP 8.5 對語言底層架構進行了大規模的「防禦性重構」，從根本上隔離了潛在的攻擊向量。

(二)原生 URI 解析引擎（Native URI Extension）：

1. 過去版本的 parse_url() 函數常因邊緣案例（Edge Cases）處理不一致，導致在解析惡意構造的 URL 時產生邏輯偏差，進而被攻擊者利用繞過安全過濾。
2. PHP 8.5 導入了完全符合 RFC 3986 與 WHATWG 標準的原生 URI 解析引擎，確保 URL 解析行為在不同環境下的絕對一致性，有效防堵了基於 URL 操縱的攻擊，如 SSRF（伺服器端請求偽造）的防禦能力大幅提升。

(三)W^X 記憶體保護機制（Write XOR Execute）：

1. PHP 8.5 在 JIT（Just-In-Time）編譯器層面引入了 W^X 記憶體保護原則。此技術確保記憶體頁面「不可同時具備寫入與執行權限」。
2. 透過此隔離手段，攻擊者即使發現了程式碼注入漏洞，也難以在記憶體中寫入並執行惡意 Shellcode，從硬體與 OS 層面阻斷了代碼執行攻擊。

(四)非對稱屬性可見性（Asymmetric Visibility）：

1. 透過 public private(set) 等修飾符，PHP 8.5 強化了物件導向中的封裝性。

2. 這不僅是語法糖，更是安全防護：它確保類別內部敏感狀態（如金鑰、配置參數）在執行過程中無法被外部任意修改，從根源上保護了應用程式的資料完整性。

二、 邏輯與型別安全之防禦深度

(一)許多網頁漏洞源於開發者對於型別處理的疏忽，PHP 8.5 透過嚴謹的語法規則，強制開發者落實安全編碼。

(二)強制型別系統的深化：

1. PHP 8.5 進一步打擊「隱式類型轉換」。例如，在陣列索引運算中若引入浮點數，系統將觸發警告。
2. 此機制能有效防止資料截斷（Data Truncation）攻擊，避免因浮點數精度誤差造成的邏輯判斷錯誤（如驗證憑證金額時的誤差繞過）。

(三)#[NoDiscard] 屬性保護：

1. 針對關鍵函數（如加密、檔案寫入、字串處理），PHP 8.5 新增 #[NoDiscard] 屬性。
2. 當開發者呼叫這些函數卻忽略其回傳值時，編譯器將拋出錯誤。
3. 這能防範因未檢測函數執行狀態（如未檢查 SQL 執行是否成功）而引發的安全漏洞。

三、 現代化密碼學與系統組件安全

針對身份驗證與會話安全，PHP 8.5 提供了更高階的抽象與防護。

(一)隨機數生成器（Randomizer）的升級：

1. 安全性極度依賴高品質的隨機性。
2. PHP 8.5 的 Randomizer 進一步整合了作業系統層級的安全隨機源（Cryptographically Secure Pseudo-Random Number Generator, CSPRNG）。
3. 這確保了產生的 Session ID、CSRF Token 或臨時密碼具有極高的不可預測性，防止預測型攻擊。

(二)Bcrypt 成本控制與資源耗盡防護：

1. 為防止攻擊者發動資源耗盡型 DoS 攻擊（透過提交巨大的成本值使伺

服器 CPU 崩潰)，PHP 8.5 對 password_hash() 引入了硬性的成本限制框架。

2. 這能確保應用程式即使在面臨高負載攻擊下，仍能保持正常運作，同時兼顧密碼儲存的安全性。

四、 廢棄高風險特性與環境治理

(一)PHP 8.5 積極推動「大掃除」，移除舊有且易受攻擊的特性，減少攻擊面：

安全維度	優化內容	安全貢獻
反序列化安全	限制 unserialize() 自動觸發行為	防止物件注入（Object Injection）造成的任意程式碼執行
輸入過濾	嚴格化 $_GET / $_POST 鍵名檢查	防範環境變數注入與非法參數擴充
變數存取	加強屬性未初始化檢查	確保程式運行狀態的可預測性，防止資訊洩漏

四、實務建議：

(一)Strict Typing 執行策略：於所有原始碼檔案首行宣告 declare(strict_types=1);，這是杜絕型別混淆漏洞最直接有效的手段。

(二)Property Hooks 取代魔術方法：使用 Property Hooks 來定義屬性的存取邏輯，不僅減少程式碼長度，更重要的是能將驗證邏輯封裝在屬性存取過程中，避免開發者忘記在特定情境下進行輸入過濾。

(三)Readonly 資料封裝：對於組態檔或識別資訊，優先使用 readonly 修飾符，確保資料在執行生命週期內具備「唯讀」屬性，從架構上消除資料被竄改的風險。

第二節 網頁設計乙級術科試題常見之弱點

一、簡介

(一)技術士技能檢定「網頁設計乙級」術科測試長期以來是台灣技職教育衡量網頁開發能力的重要指標。然而，該試題設計之初衷主要著重於功能實作（如 CRUD 資料處理、分頁邏輯、圖片上傳）與排版美化，對於當前 Web 應用程式安全（Web Application Security）的防禦規範，仍存在一定的理論與實踐落差。

(二)乙級試題之架構與技術堆疊

1. 根據勞動部勞動力發展署公布之術科試題（附錄 1)，考生需在規定時間內完成四個網站系統（如：校園社群網站、電影訂票系統等)。
2. 其技術核心主要包含：
   (1) 前端技術： HTML5、CSS3、JavaScript（jQuery)。
   (2) 後端語言： PHP（多數教學現場採用 PHP 7.x 或 8.x)。
   (3) 資料庫： MySQL/MariaDB。

(三)學術研究與實務落差之價值

1. 目前職訓現場多以「達成功能」為首要目標，導致許多取得證照的開發者在進入產業後，仍編寫出具備結構性弱點的程式碼。
2. 本研究透過 OWASP ZAP 對這四個標準試題網站進行自動化掃描，其意義在於：
   (1) 具體化風險： 將模糊的資安威脅轉化為具體的掃描報告與損害等級分析。
   (2) 標準化強化： 針對試題中重複出現的邏輯（如 login.php、update.php)，建立一套可重複使用的 PHP 安全修復範本。
3. 網頁設計乙級試題雖具備完整的 Web 功能結構，但在資安防護上仍有顯著的強化空間。這為本研究後續使用 OWASP ZAP 進行「漏洞檢測」與「程式修復」提供了具備代表性且具教學價值的測試樣本。

二、網站安全漏洞掃描測試結果

Alert type	Risk	Count
SQL Injection - MySQL	High	2 (8.7%)
SQL Injection - MySQL (Time Based)	High	1 (4.3%)
Absence of Anti-CSRF Tokens	Medium	2 (8.7%)
Content Security Policy (CSP) Header Not Set	Medium	5 (21.7%)
Directory Browsing	Medium	3 (13.0%)
Missing Anti-clickjacking Header	Medium	5 (21.7%)
Parameter Tampering	Medium	4 (17.4%)
Vulnerable JS Library	Medium	1 (4.3%)
Cookie No HttpOnly Flag	Low	1 (4.3%)
Cookie without SameSite Attribute	Low	1 (4.3%)
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	5 (21.7%)
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	5 (21.7%)
X-Content-Type-Options Header Missing	Low	5 (21.7%)

三、安全漏洞說明

（一） SQL Injection:

1. 簡介（Introduction）

（1）定義： SQL Injection 是一種發生在應用程式資料層的安全漏洞。當程式未對用戶輸入的資料進行適當過濾或轉義，並將其直接拼接至 SQL 查詢語句中時，攻擊者可藉此改變原有的查詢邏輯。

（2）核心問題： 程式無法區分「開發者預期的指令」與「用戶輸入的惡意資料」，導致資料庫執行了未經授權的指令。

（3）影響範圍： 攻擊者可藉此繞過身份驗證、竊取敏感資料（如用戶密碼、個人資料）、篡改或刪除資料庫數據，甚至在某些配置下取得伺服器的作業系統控制權。

2. 範例（Example Scenario）

（1）假設情境： 一個簡單的後台登入系統，透過 username 和 password 驗證用戶。

（2）正常請求： 用戶輸入 admin 和 password123。

SQL 語句：SELECT * FROM users WHERE username = 'admin' AND password = 'password123';

結果：資料庫回傳該用戶資料，登入成功。

（3）攻擊輸入： 攻擊者在帳號欄位輸入 ' OR '1'='1，密碼隨意輸入。

邏輯篡改： 最終生成的 SQL 變為：

SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '...';

（4）觸發攻擊： 由於 '1'='1' 恆為真（True），且 OR 運算子的優先級使條件成立，伺服器將回傳資料表中第一個用戶（通常是管理員）的資料，攻擊者無需密碼即可登入。

3. 危險寫法（Vulnerable Code - PHP）

以下是使用 mysqli 擴充套件但直接拼接字串的典型錯誤示範：

```
// login.php
$conn = mysqli_connect("localhost", "db_user", "db_pass",
```

```
"database");
$username = $_POST['username'];
$password = $_POST['password'];
// 危險：直接將變數拼接到 SQL 字串中
$query = "SELECT * FROM users WHERE username = '$username' AND
password = '$password'";
$result = mysqli_query($conn, $query);
if (mysqli_num_rows($result) > 0) {
    echo "登入成功！";
} else {
    echo "帳號或密碼錯誤。";
}
```

4. 安全寫法 (Secure Code - PHP)

最有效的防禦方式是使用 預編譯陳述式 (Prepared Statements)與參數化查詢 (Parameterized Queries)。推薦做法：使用 PHP Data Objects (PDO)

```
// secure_login.php
try {
    $pdo = new PDO("mysql:host=localhost;dbname=database",
    "db_user", "db_pass");
// 步驟 A：編寫具備佔位符 (?) 的 SQL 模板
    $sql = "SELECT id, username FROM users WHERE username = ?
AND password = ?";
    $stmt = $pdo->prepare($sql);
// 步驟 B：綁定參數並執行（資料與指令分離）
// 密碼建議使用 password_verify 驗證，此處僅演示參數化查詢
    $stmt->execute([$_POST['username'], $_POST['password']]);
    $user = $stmt->fetch();
```

```
        if ($user) {
        echo "登入成功，歡迎 " .htmlspecialchars($user['username']);
        } else {
            echo "認證失敗。";
        }
    } catch (PDOException $e) {
        die("資料庫錯誤。");
    }
```

5. 開發建議 (Development Recommendations)

(1) 從資訊管理與系統架構的角度，防禦 SQL Injection 應遵循「深度防禦」原則：

- 全面參數化 (Full Parameterization)： 所有的資料庫互動（包含 SELECT, INSERT, UPDATE, DELETE）皆必須使用預編譯陳述式。這是目前公認最有效的防線。
- 輸入驗證 (Input Validation)： 採取「白名單」策略。例如，若預期輸入是數字（如 user_id)，應在進入資料庫前強制轉換型別 (int)$_POST['id']。
- 最小權限原則 (Principle of Least Privilege)： 網頁應用程式連接資料庫的帳號，不應具備 DROP TABLE 或存取系統表（如 information_schema）的權限。應針對不同模組分配僅具備所需權限的帳號。

(2) 避免詳細錯誤訊息： 在正式環境中，應關閉資料庫錯誤顯示（如 display_errors = Off)，避免攻擊者透過錯誤回顯 (Error-based SQLi）推測資料庫結構。

(3) 使用 ORM 工具： 導入如 Eloquent (Laravel) 或 Doctrine 等 ORM 框架，這些工具底層預設即使用參數化查詢，能大幅降低人為疏忽導致的漏洞。

（二） SQL Injection – MySQL (Time Based)

1. 簡介 (Introduction)

   (1) 這是一種更隱蔽、更耗時的 盲注 (Blind SQL Injection) 技術。

   (2) 當應用程式-配置得很安全，不會在前端顯示錯誤訊息 (Error-based)，也不會因為查詢成功或失敗而改變頁面內容 (Boolean-based) 時，駭客就會改用時間盲注

   (3) (Time-Based)。其核心原理是：利用 SQL 的時間延遲函數，根據網頁「回應速度」的快慢，來推斷資料庫中的資訊。

2. 攻擊範例

   範例場景：猜測資料庫名稱的第一個字母，假設駭客想知道資料庫名稱的第一個字母是不是 'a'：

   AND (SELECT IF(SUBSTRING(database(),1,1)='a', SLEEP(5), 1))

   資料庫執行的邏輯：

   SUBSTRING(database(),1,1)='a'：檢查資料庫名稱的第一個字。如果是 'a'：執行 SLEEP(5)。伺服器會在 5 秒後才把網頁傳回給駭客。如果不是 'a'：執行 1 (立即回傳)。伺服器會秒回。 駭客的判斷：如果我等了 5 秒才看到網頁 $\rightarrow$ 猜對了，第一個字是 'a'。如果網頁瞬間出現 $\rightarrow$ 猜錯了，換下一個字母 'b' 繼續試。

3. 危險寫法 (容易受 Time-based 攻擊)

```
$id = $_GET['id'];
$sql = "SELECT name FROM products WHERE id = $id";
$result = $conn->query($sql);
```

4. 安全寫法 (使用 PDO 預處理)

```
$id =  $_GET['id'];
$stmt = $pdo->prepare("SELECT name FROM products WHERE id = ?");
$stmt->execute([$id]); //輸入包含 SLEEP()，也會被當作純字串處理
```

5. 開發建議：

(1) 設定請求超時:在網頁伺服器或 PHP 層級設定合理的 Timeout，防止資料庫被長延遲攻擊掛起。

(2) 監控慢查詢 (Slow Logs)：如果發現大量帶有 SLEEP 或異常延遲的 SQL 記錄，通常就是受攻擊的徵兆。

(三) Absence of Anti-CSRF tokens

1. 簡介 (Introduction)

(1) 跨站請求偽造 (Cross-Site Request Forgery, CSRF) 是一種迫使已驗證用戶在當前 Web 應用程式上執行非自願操作的攻擊手法。

(2) 其核心問題在於瀏覽器在發送請求時，會自動附帶與該域名相關的 Cookie (如 Session ID)。

(3) 若 Web 表單或 API 接口欠缺 Anti-CSRF Token (防護令牌)，攻擊者即可利用社交工程手段引導受害者訪問惡意網站，藉由受害者的登入狀態發送偽造請求。由於伺服器僅依賴 Cookie 進行身份驗證，無法區分該請求是由用戶自主觸發，還是由第三方網站惡意偽造。

2. 範例 (Example Scenario)

假設某銀行網站的轉帳功能僅依賴 Session Cookie，且沒有 Token 保護：

(1) 正常請求： 用戶登入 bank.com 後，點擊轉帳，瀏覽器發送 POST /transfer?to=userB&amount=1000。

(2) 攻擊誘餌：攻擊者建構一個惡意網頁，內含隱藏表單或 JavaScript 自動執行請求：

```
<form id="attack" action="https://bank.com/transfer"
method="POST">
   <input type="hidden" name="to" value="attacker" />
   <input type="hidden" name="amount" value="10000" />
</form>
<script>document.getElementById('attack').submit();</script>
```

(3) 觸發攻擊： 當受害者在同一瀏覽器開啟此惡意網頁時，瀏覽器會自動

帶上 bank.com 的 Session Cookie，銀行伺服器誤以為是合法操作，導致資金被竊。

3. 危險寫法（Vulnerable Code - PHP）

以下是一個典型缺乏 CSRF 防護的 PHP 處理邏輯，僅檢查用戶是否登入：

```
// transfer.php
session_start();
// 僅驗證 Session，未驗證請求來源的唯一性
if (isset($_SESSION['user_id'])) {
    $to = $_POST['to'];
    $amount = $_POST['amount'];

    // 執行轉帳邏輯（此處極易受 CSRF 攻擊）
    update_balance($_SESSION['user_id'], $to, $amount);
    echo "轉帳成功！";
}
```

4. 安全寫法（Secure Code - PHP）

正確的做法是在表單中加入一個隨機、不可預測且具時效性的 CSRF Token：

(1) 步驟 A：生成並儲存 Token

```
session_start();
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

(2) 步驟 B：在 HTML 表單中嵌入 Token

```
<form action="transfer.php" method="POST">
    <input type="hidden" name="csrf_token" value="<?php echo
$_SESSION['csrf_token']; ?>">
    接收帳號：<input type="text" name="to">
```

```
    金額：<input type="text" name="amount">
    <button type="submit">確認轉帳</button>
</form>
```

(3) 步驟 C：伺服器端驗證 Token

```
session_start();
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['csrf_token']) ||
$_POST['csrf_token'] !== $_SESSION['csrf_token']) {
        die("CSRF 攻擊檢測：無效的請求來源。");
    }
    // 驗證通過，執行後續邏輯
}
```

5. 開發建議 (Development Recommendations)

從資管與軟體工程角度出發，建議採取以下多層次防禦策略：

(1) 同步令牌模式 (Synchronizer Token Pattern)： 這是最標準的做法，確保每個狀態更改請求（POST/PUT/DELETE）都包含伺服器生成的隨機 Token。

(2) 使用現代框架： 建議開發者使用如 Laravel、Symfony 或 Django 等內建 CSRF 防護機制的框架，這些框架會自動處理 Token 的生成與校驗。

(3) SameSite Cookie 屬性： 將 Session Cookie 的屬性設置為 SameSite=Lax 或 Strict。這能限制瀏覽器在跨站請求時發送 Cookie，從根本上減少攻擊面。

(4) 區分請求方法： 嚴格遵守 RESTful 規範，確保 GET 請求僅用於資料讀取而不產生副作用（不改變資料庫狀態）。

(5) 雙重驗證 (Two-Factor Auth)： 對於高風險操作（如修改密碼、大額轉帳），除了 Token 之外，應增加簡訊驗證碼或密碼二次確認，作為最後一道防線。

（四） Content Security Policy(CSP) Header Not Set

1. 簡介（Introduction）

（1）內容安全策略（Content Security Policy, CSP）是一種強大的安全性標頭（HTTP Response Header），旨在防範跨站腳本攻擊（XSS）、點擊劫持（Clickjacking）及數據注入攻擊。

（2）其核心機制在於透過宣告一組白名單策略，告訴瀏覽器哪些來源的資源（如 JavaScript、CSS、圖片、框架）是受信任的。

（3）若 Web 應用程式未設置 CSP Header，瀏覽器將預設執行所有載入的腳本。這意味著一旦駭客成功注入惡意代碼（例如透過 URL 參數或留言板），瀏覽器將無法分辨該腳本是來自原始伺服器還是攻擊者，從而導致用戶憑證（Cookie）被盜取或頁面內容被篡改。

2. （範例（Example Scenario）

假設某電子商務網站的搜尋功能存在 XSS 漏洞，且未配置 CSP：

（1）攻擊路徑： 攻擊者誘使受害者點擊包含惡意腳本的連結：

https://shop.com/search?q=<script>fetch('https://evil.com/steal?cookie=' + document.cookie)</script>

（2）瀏覽器行為： 由於缺乏 CSP 限制，瀏覽器會毫不懷疑地執行 fetch 指令，將受害者的 Session Cookie 發送到攻擊者的伺服器。

（3）防護效果： 若網站設置了 Content-Security-Policy: default-src 'self';，瀏覽器在解析到該腳本時，會發現其試圖向非同源（non-self）的 evil.com 發送資料，進而主動攔截該請求並在開發者控制台報錯。

3. 危險寫法（Vulnerable Code - PHP）

以下 PHP 程式碼僅輸出了內容，完全沒有定義任何安全響應頭，導致瀏覽器暴露在風險中：

```
<?php
// index.php
session_start();
```

```
// 缺乏安全性標頭的配置
header("Content-Type: text/html; charset=utf-8");
echo "<h1>歡迎來到我的網站</h1>";
echo "<p>搜尋結果: " . $_GET['q'] . "</p>"; // 此處極易遭受反射型 XSS 攻擊
?>
```

4. 安全寫法 (Secure Code - PHP)

正確的做法是在頁面輸出任何內容之前，透過 header() 函數發送嚴謹的 CSP 策略。

(1) 步驟 A：設置基礎防禦策略（僅允許同源資源）

```
<?php
// 強制瀏覽器僅能執行來自同一個 Domain 的腳本與資源
header("Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';");
?>
```

(2) 步驟 B：進階防禦（使用 Nonce 隨機值防止 Inline Script 攻擊）

對於必須在 HTML 中內嵌 JavaScript 的情況，應使用 Nonce：

```
<?php
session_start();
// 生成隨機 Nonce
if (empty($_SESSION['csp_nonce'])) {
    $_SESSION['csp_nonce'] = bin2hex(random_bytes(16));
}
$nonce = $_SESSION['csp_nonce'];
// 發送包含 Nonce 的 CSP 標頭
header("Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-$nonce';");
?>
```

```
<script nonce="<?php echo $nonce; ?>">
    console.log("此安全腳本已獲得授權執行。");
</script>
<script>
    alert("此腳本沒有 Nonce，將被瀏覽器攔截！");
</script>
```

5. 開發建議 (Development Recommendations)

從資管與資訊安全架構角度出發，建議採取以下防禦策略：

(1) 採行最小權限原則 (Principle of Least Privilege)： 初始配置應使用 default-src 'none';，再依據需求逐一開放必要的資源來源（如 img-src 或 font-src)。

(2) 禁用不安全的關鍵字： 嚴禁在生產環境使用 'unsafe-inline'（允許內嵌腳本）與 'unsafe-eval'（允許執行字串代碼)，這兩者會使 CSP 的防禦力大幅下降。

(3) 導入報告機制 (Report-Only Mode)： 在正式實施前，可先使用 Content-Security-Policy-Report-Only 標頭。這能讓開發者在不影響用戶使用的情況下，收集違反策略的報告，用以調整策略。

(4) 防止點擊劫持 (Clickjacking)： 在 CSP 中加入 frame-ancestors 'none'; 或 'self';，這比傳統的 X-Frame-Options 標頭更具彈性且強大。

(5) 自動化審計工具： 建議定期使用 Google 的 CSP Evaluator 等工具，檢查策略是否存在邏輯漏洞（如過度寬鬆的白名單)。

（五） Directory Browsing

1. 簡介 (Introduction)

(1) 目錄遍歷 (Directory Browsing) 是一種 Web 伺服器配置錯誤，當使用者請求一個不包含預設首頁檔案（如 index.php 或 index.html）的目錄時，伺服器會自動列出該目錄下的所有檔案與子目錄。

(2) 其核心問題在於伺服器端的權限控管過於寬鬆，未對自動生成索引

（Auto-indexing）功能進行限制。

(3) 攻擊者可藉此輕鬆查閱網站的檔案結構，進而發現備份檔案如 .bak、.zip)、組態設定檔（如 config.php.dist)、敏感日誌或尚未公開的原始碼。這通常是執行進階攻擊（如滲透測試中的偵察階段）的重要資訊來源。

2. 範例 (Example Scenario)

假設某開發者將資料庫連接資訊存放在 /config 目錄下，且該目錄沒有首頁檔案：

(1) 正常訪問： 使用者訪問 https://example.com/index.php 看到正常網頁。

(2) 漏洞探測： 攻擊者嘗試訪問 https://example.com/uploads/ 或 https://example.com/config/。

(3) 觸發洩漏： 伺服器回傳一個名為 "Index of /config" 的頁面，展示如下內容：

[DIR] Parent Directory

[FILE] db_connection.php.bak

[FILE] secret_key.txt

攻擊擴大： 攻擊者下載 .bak 檔案，取得資料庫帳號密碼，最終導致整個資料庫被竊取。

3. 危險寫法 (Vulnerable Configuration)

目錄遍歷通常不是 PHP 程式碼本身的問題，而是伺服器環境配置（Apache/Nginx）的疏忽。以下是常見的危險配置：

(1) Apache (.htaccess 或 httpd.conf):

```
# 關鍵字 'Indexes' 會啟用目錄列表功能
Options +Indexes +FollowSymLinks
```

(2) Nginx (nginx.conf):

```
location / {
# 將 autoindex 設置為 on 會開啟目錄遍歷
```

```
        autoindex on;
    }
```

4. 安全寫法（Secure Mitigation）

要修復此問題，需從伺服器層級關閉自動索引功能，或在應用程式層級進行防禦。

(1) 方法 A：修改伺服器配置（推薦）

- 在 Apache 的 .htaccess 中明確禁用索引：

  Options -Indexes

- 在 Nginx 中，應確保 autoindex 設為 off。

(2) 方法 B：放置空首頁檔案（基礎防禦）

在每個不希望被遍歷的目錄下放置一個空的 index.php 或 index.html。當伺服器發現該檔案存在時，便不會列出目錄清單。

```
<?php
// index.php
// 重導向回首頁或顯示 403 錯誤
header("HTTP/1.1 403 Forbidden");
exit("Access Denied");
?>
```

(3) 方法 C：PHP 權限檢查（動態防禦）

如果目錄記憶體放的是使用者上傳的檔案，應透過 PHP 腳本控制存取，而非直接開放目錄路徑：

```
// download.php
session_start();
if (!isset($_SESSION['user_id'])) {
    die("未授權存取");
}
// 透過腳本讀取檔案流，隱藏真實檔案系統路徑
readfile("/secure_storage/user_files/" . $_GET['file']);
```

5. 開發建議（Development Recommendations）

從資管與系統管理維度出發，建議採取以下多層次防禦策略：

(1) 最小化暴露面（Attack Surface Reduction）： 確保生產環境（Production）中所有 Web 伺服器的 Auto-indexing 功能皆已關閉。這是資安合規性檢查（如 ISO 27001）的基本要求。

(2) 檔案命名規範與路徑隱蔽： 敏感檔案不應存放在 Web Root（網頁根目錄）下。建議將設定檔與備份檔移動至瀏覽器無法直接存取的目錄。

(3) 定期執行弱點掃描： 利用掃描工具（如您論文中提到的 OWASP ZAP 或 Nikto）自動探測網站目錄是否存在自動索引漏洞。

(4) 自定義錯誤頁面： 配置伺服器當發生 403（Forbidden）或 404（Not Found）時顯示統一的錯誤頁面，避免洩漏伺服器版本（如 Apache/2.4.41）等額外技術資訊。

(5) 嚴格的權限管理（ACL）： 針對存放敏感資料的目錄，應從作業系統層級（Linux Permissions）限制 Web 服務帳號（如 www-data）僅具有必要的讀寫權限。

（六） Missing Anti-clickjacking Header

1. 簡介（Introduction）

(1) 點擊劫持（Clickjacking），又稱「使用者介面紅利攻擊（UI Redressing），是一種欺騙性的技術，攻擊者將受信任的目標網站嵌入到惡意網站的透明 iframe 中。

(2) 其核心問題在於網頁允許被第三方域名嵌套，攻擊者會在透明的目標網頁上方覆蓋一層誘騙內容（如「點擊領取獎品」）。

(3) 當用戶點擊看似無害的惡意網頁按鈕時，實際上觸發的是底層被隱藏目標網站上的敏感操作（如：刪除帳號、修改設定、發送隱私訊息）。由於操作是在受害者的登入狀態下進行，伺服器會視為合法行為。

2. 範例（Example Scenario）

假設網站的「刪除帳號」頁面（account.php?action=delete）缺乏防護：

(1) 攻擊誘餌： 攻擊者製作一個吸引點擊的網頁，例如「恭喜！您獲得了最新 iPhone，點擊此處領取」。

(2) 透明嵌套： 攻擊者使用 CSS 將網站的刪除按鈕精確地重疊在「領取獎品」按鈕的正上方，並將社交網站的 opacity 設為 0（完全透明）。

(3) 觸發攻擊： 當受害者點擊「領取獎品」時，瀏覽器實際上點擊了社交網站的「確認刪除帳號」按鈕。因此在毫不知情的情況下失去了帳號。

3. 危險寫法 (Vulnerable Code - PHP)

若 PHP 程式碼中未對 HTTP 響應頭進行限制，預設情況下瀏覽器允許該網頁被任何網站嵌入：

```
<?php
// settings.php
session_start();
// 僅驗證身份，但未限制 UI 顯示權限
if (!isset($_SESSION['user_id'])) {
    die("請先登入");
}
echo "<h1>個人設定</h1>";
echo "<button onclick='deleteAccount()'>刪除帳號</button>";
// 缺乏 X-Frame-Options 或 CSP frame-ancestors 標頭
?>
```

4. 安全寫法 (Secure Code - PHP)

正確的做法是透過伺服器端發送標頭，限制網頁被框架化(Framing)的權限。

(1) 步驟 A：使用 X-Frame-Options 標頭

這是目前最普遍且兼容性最佳的傳統做法。

```
<?php
// 禁止所有網站（包括同源網站）將此頁面嵌入 iframe
header("X-Frame-Options: DENY");
```

```
// 或：僅允許同源網站嵌入（通常用於網站內部的功能嵌套）
// header("X-Frame-Options: SAMEORIGIN");
?>
```

(2) 步驟 B：使用 Content Security Policy (CSP) 之 frame-ancestors

這是現代資安標準中更強大且精確的替代方案，可取代 X-Frame-Options。

```
<?php
// 限制只有自己的網站可以嵌套此頁面
header("Content-Security-Policy: frame-ancestors 'self';");

// 若需允許特定合作夥伴域名嵌套：
// header("Content-Security-Policy: frame-ancestors 'self'
https://trusted-partner.com;");
?>
```

5. 開發建議 (Development Recommendations)

從資管與資訊安全治理的角度出發，建議採取以下多層次防禦策略：

(1) 優先採用 DENY 策略： 對於包含敏感操作（轉帳、刪除、修改密碼）的頁面，應強制設定為 DENY，完全禁止被嵌入。

(2) 統一標頭管理：建議在 Web 伺服器層級（如 Apache 的.htaccess 或 Nginx 的 nginx.conf）全域配置此標頭，確保開發者不會因為遺漏個別 PHP 檔案而產生漏洞。

(3) 淘汰舊式 Frame-busting 腳本： 過去常使用 JavaScript（如 if (top != self) ...）來檢測嵌套，但這類方法容易被攻擊者繞過，應改為依賴標準的 HTTP 安全標頭。

(4) 縱深防禦機制： 點擊劫持通常與 CSRF 結合使用，因此應同時實施 AntiCSRF Tokens，即使攻擊者成功誘導用戶點擊，也會因為缺乏 Token 而導致請求失敗。

(5) 資安意識教育： 從資管角度看，技術防禦是基礎，但教育使用者識別社交工程陷阱（不隨意點擊不明連結）同樣是降低點擊劫持風險的重要環節。

（七） Paramater Tampering

1. 簡介（Introduction）

(1) 參數篡改（Parameter Tampering）是一種利用 Web 應用程式過度信任客戶端輸入資料的攻擊手段。攻擊者透過修改 HTTP 請求中的參數（如 URL 查詢字串、表單欄位、Cookie 或隱藏表單內容），試圖改變應用程式的執行結果。

(2) 其核心問題在於缺乏伺服器端驗證。開發者往往假設用戶端回傳的資料（特別是隱藏欄位）與伺服器當初發出的資料一致。

(3) 攻擊者不需高深的技術，僅需透過瀏覽器開發者工具（F12）或代理工具（如 Burp Suite）即可修改商品價格、用戶權限等級或檔案存取路徑。

2. 範例（Example Scenario）

假設某電商網站的結帳流程，將商品單價存放在隱藏表單欄位中：

(1) 正常操作： 用戶選購一件 5,000 元的產品，點擊結帳，瀏覽器發送：POST /checkout.php，內容含 item_id=101&price=5000&qty=1。

(2) 攻擊行為： 攻擊者在點擊結帳前，使用開發者工具將 HTML 中的隱藏欄位值改為 1：

HTML

```
<input type="hidden" name="price" value="1">
```

觸發攻擊： 伺服器接收到請求後，若未重新從資料庫比對價格，便會以 1 元的價格完成交易，造成企業重大的財務損失。

3. 危險寫法（Vulnerable Code - PHP）

以下程式碼直接信任來自 POST 請求的價格資訊，這在資安實務上是非常危險的：

```
// checkout.php
session_start();
if (isset($_POST['item_id']) && isset($_POST['price'])) {
    $item_id = $_POST['item_id'];
    $price = $_POST['price']; // 直接信任前端傳來的價格
    $qty = $_POST['qty'];
    $total = $price * $qty;
    // 執行扣款邏輯
    process_payment($_SESSION['user_id'], $total);
    echo "付款成功，總金額：$" . $total;
}
```

4. 安全寫法（Secure Code - PHP）

正確的做法是**「永遠不要信任用戶端的輸入」**。所有關鍵數據（如價格、權限、折扣）應一律以伺服器端資料庫為準。

（1）步驟 A：僅接收 ID，從後端查表

```
session_start();
if (isset($_POST['item_id'])) {
    $item_id = (int)$_POST['item_id'];
    $qty = (int)$_POST['qty'];
// 關鍵：從資料庫獲取真實價格，而非從 $_POST 獲取
    $stmt = $pdo->prepare("SELECT price FROM products WHERE
id = ?");
    $stmt->execute([$item_id]);
    $product = $stmt->fetch();
    if ($product) {
```

```
        $real_price = $product['price'];
        $total = $real_price * $qty;
        // 驗證庫存與執行扣款
        process_payment($_SESSION['user_id'], $total);
    }
}
```

(2) 步驟 B：對敏感參數進行數位簽章 (Optional)

若某些情境必須在前端傳遞狀態，可使用雜湊值 (HMAC) 驗證資料是否被改動：

```
// 生成時
$mac = hash_hmac('sha256', $data_string, $secret_key);
// 驗證時
if (!hash_equals($mac, $_POST['mac'])) {
    die("偵測到參數非法篡改！");
}
```

5. 開發建議 (Development Recommendations)從資管與軟體品質保證的角度出發，建議採取以下多層次防禦策略：

(1) 後端權威原則： 涉及金流、權限或業務邏輯的數據，必須在伺服器端進行二次校驗。前端傳來的參數應僅視為「意圖」，而非「事實」。

(2) 輸入驗證與過濾 (Input Validation)： 嚴格檢查參數的資料類型（如數字、字串）、長度與格式。例如數量 (qty) 不應出現負數。

(3) 使用 Session 存儲狀態： 敏感的暫時性資料（如購物車總額、結帳進度）應存放在伺服器端的 $_SESSION 中，而非暴露在 HTML 隱藏欄位或 Cookie 裡。

(4) 最小權限存取控制： 確保用戶無法透過修改 URL 參數（如 user_id=100 改為 user_id=1）存取他人的資料（這也稱為 IDOR 漏洞）。

(5) 日誌紀錄與異常監控： 對於不合理的參數變動（例如商品價格與資料庫落差極大時）應觸發系統警報，以便資安人員即時介入調查。

（八） Vulnerable JS Library（使用具安全性弱點的 JS 函式庫）

1. 簡介（Introduction）

(1) Vulnerable JS Library 是指網頁應用程式引用了已知存在安全漏洞（CVE）的第三方 JavaScript 函式庫（如舊版的 jQuery、Bootstrap 或 Handlebars）。

(2) 其核心問題在於缺乏依賴管理與更新機制。隨著開源技術的演進，許多舊版函式庫被發現存在跨站腳本攻擊（XSS）、原型污染（Prototype Pollution）或點擊劫持等漏洞，並已公開在 CVE 或 NPM 漏洞資料庫中。

(3) 攻擊者可以透過掃描工具辨識目標網站所使用的 JS 版本，並針對該版本的已知弱點進行精確打擊。由於這些漏洞存在於底層工具中，即使開發者的業務邏輯代碼寫得再安全，也可能因為引用的元件而導致整體防線潰敗。

2. 範例（Example Scenario）

漏洞背景： 假設某網站為了快速開發，引用的 jQuery 版本為極老舊的 1.10.2。

(1) 已知弱點： 該版本在處理選擇器（Selector）時存在 XSS 漏洞，攻擊者可以透過特定格式的字串繞過過濾。

(2) 攻擊手法： 攻擊者建構一個帶有惡意 Payload 的 URL：

https://example.com/#<img src=x onerror=alert('XSS_Vulnerability')>

(3) 觸發攻擊： 舊版 jQuery 在解析 location.hash 時，會錯誤地執行該 HTML 標籤內的 JavaScript 代碼。若受害者點擊連結，攻擊者即可竊取其 Session 或執行未經授權的操作。

3. 危險寫法 (Vulnerable Code - HTML/PHP)

以下是在 PHP 模板中常見的「一勞永逸」式引入，開發者往往在專案上線後就再也沒有檢查過版本：

HTML

```
<html>
<head>
    <title>我的系統</title>
    <script src="https://code.jquery.com/jquery-
1.10.2.min.js"></script>
    <script
src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.0/js/bootst
rap.min.js"></script>
</head>
<body>
    <h1>系統內容</h1>
    <script>
        // 舊版 jQuery 對 $(location.hash) 的處理存在漏洞
        $(location.hash);
    </script>
</body>
</html>
```

4. 安全寫法 (Secure Code - Management)

解決此類問題主要不在於「程式邏輯」，而是在於「版本控管」與「資源引用方式」。

(1) 步驟 A：引用穩定且更新的版本

應切換至官方維護中的長期支援版本 (LTS)，並使用子資源完整性 (SRI) 驗證：

HTML

```
<script
    src="https://code.jquery.com/jquery-3.7.1.min.js"
    integrity="sha256-
/JqT3SQfawRcv/BIHPThkBvs0OEvtFFmqPF/lYI/Cxo="
    crossorigin="anonymous">
</script>
```

(2) 步驟 B：使用依賴管理工具 (npm/composer)

不要手動下載檔案，而是透過工具自動化管理，這能讓你輕鬆追蹤哪些元件需要修補。

```
Bash
# 使用 npm 檢查專案中已知的漏洞
npm audit
```

5. 開發建議 (Development Recommendations)

從資管與資訊安全架構角度出發，建議採取以下多層次防禦策略：

(1) 建立軟體材料清單 (SBOM)： 記錄專案中所使用的所有第三方組件及其版本。這是資管系統審核 (Audit) 與合規性的基礎。

(2) 啟用自動化弱點掃描： 建議在 CI/CD 流程中整合自動化工具 (如 GitHub Dependabot 或 Snyk)，當依賴項出現漏洞時，系統會自動發出警報並建議修復版本。

(3) 使用子資源完整性 (SRI)： 當從 CDN 引用資源時，務必包含 integrity 雜湊值。這能確保即使 CDN 伺服器被駭，惡意代碼也不會在用戶的瀏覽器中執行。

(4) 實施 CSP 指令： 透過 script-src 限制僅能載入來自受信任網域或特定雜湊值的腳本，縮小攻擊面。

(5) 虛擬補丁 (Virtual Patching)： 若因系統架構過於老舊而無法立即升級函式庫，可考慮在 Web 應用程式防火牆 (WAF) 上設定過濾規則，攔截已知的針對性攻擊流量。

（九） Cookie No HttpOnly Flag

1. 簡介（Introduction）

(1) HttpOnly 是一個附加在 HTTP 響應頭 Set-Cookie 中的屬性標籤。當 Cookie 被設置為 HttpOnly 時，瀏覽器會限制客戶端腳本（如 JavaScript）透過 document.cookie API 存取該 Cookie。

(2) 其核心問題在於身份憑證的暴露。預設情況下，瀏覽器允許腳本存取 Cookie。如果 Web 應用程式存在 XSS 漏洞，攻擊者可以注入惡意腳本來讀取包含 Session ID 的 Cookie。

(3) 若欠缺 HttpOnly 標籤，攻擊者一旦成功執行 XSS，即可輕易竊取用戶的會話令牌（Session Token），並在自己的瀏覽器中冒充該用戶登入系統，這被稱為會話劫持（Session Hijacking）。

2. 範例（Example Scenario）

(1) 漏洞前提： 假設某 PHP 網站未開啟 HttpOnly，且存在一個簡單的反射型 XSS 漏洞。

(2) 攻擊路徑： 攻擊者引導受害者訪問以下連結：

https://vulnerable-site.com/search.php?name=<script>new Image().src="https://evil.com/steal?c="+document.cookie;</script>

(3) 觸觸發攻擊

- 受害者的瀏覽器執行了惡意腳本。
- 由於 Cookie 沒有 HttpOnly 保護，document.cookie 成功讀取了 PHPSESSID=abc123...。.
- 腳本將此敏感資料傳送至攻擊者的伺服器

(4) 劫持後果： 攻擊者取得 Session ID 後，直接手動設置自己瀏覽器的 Cookie，即可在無需帳號密碼的情況下進入受害者的後台介面。

3. 危險寫法（Vulnerable Code - PHP）

以下是 PHP 常見的危險配置方式，預設或手動設置時忽略了安全性標籤：

(1) // 方法 A：直接使用預設 session_start()，且 php.ini 未配置安全選項

```
session_start();
```

(2) // 方法 B：手動設置 Cookie 時忽略了 HttpOnly 參數

```
setcookie("user_auth", "token_value", time()+3600, "/");
// 參數清單中最後兩個 boolean 通常代表 Secure 與 HttpOnly，此處皆為預設的 false
```

4. 安全寫法（Secure Code - PHP）

正確的做法是在生成 Cookie 時，明確要求瀏覽器禁止腳本存取。

(1) 步驟 A：全局配置（推薦）

在 php.ini 或程式入口點（如 config.php）進行全局設定，確保所有 Session Cookie 自動具備防護：

```
// 在 session_start() 之前設置
session_start([
    'cookie_httponly' => true,
    'cookie_secure'   => true,   // 僅透過 HTTPS 傳輸
    'samesite'        => 'Lax'   // 防範 CSRF
]);
```

(2) 步驟 B：手動設置特定 Cookie

使用 setcookie 函數時，將第七個參數設置為 true：

```
$name = "user_preference";
$value = "dark_mode";
$expire = time() + 3600;
$path = "/";
$domain = "";
$secure = true;   // 僅 HTTPS
$httponly = true; // 關鍵：禁止 JavaScript 存取
setcookie($name, $value, $expire, $path, $domain, $secure, $httponly);
```

5. 開發建議 (Development Recommendations)

從資管與系統合規性的角度出發，建議採取以下防禦策略：

(1) 深度防禦 (Defense in Depth)： HttpOnly 並不能防止 XSS 攻擊本身，但它能極大地降低 XSS 發生後的損害（防止 Session 被盜）。應與 CSP 策略、輸入過濾並行。

(2) 強制 Secure 屬性： 除了 HttpOnly，對於身份憑證應同時開啟 Secure 標籤，確保 Cookie 不會在不安全的 HTTP 環境下以明文傳輸。

(3) 定期審計配置： 使用瀏覽器開發者工具的 "Application" 面板或 OWASP ZAP 掃描，檢查所有敏感 Cookie 的 HttpOnly 欄位是否已被勾選。

(4) Session 過期機制： 縮短 Session 的有效時間（TTL），並在用戶登入/登出時重新生成 Session ID (session_regenerate_id())，降低憑證被利用的窗口期。

(5) 使用現代 Session 管理： 盡可能依賴 PHP 內建的 Session 機制並正確配置，避免自行實作不具備安全標籤的 Cookie 認證系統。

（十） Cookie without SameSite Attribute(Cookie 欠缺 SameSite 屬性)

1. 簡介 (Introduction)

(1) SameSite 屬性是 HTTP 響應頭 Set-Cookie 中的一個安全性指令，用於告知瀏覽器是否應在跨站請求（Cross-Site Requests）中附帶該 Cookie。

(2) 其核心問題在於跨站請求的隱性信任。若未顯式設置 SameSite 屬性，舊版瀏覽器預設會隨所有請求發送 Cookie。這使得攻擊者可以利用受害者的登入狀態，在第三方網站發起惡意請求（即 CSRF 攻擊）。

(3) 透過配置 SameSite 屬性，開發者可以精確控制 Cookie 的傳輸行為，進而從瀏覽器底層阻斷大多數非預期的跨站請求，強化用戶會話的安全性與隱私保護。

2. 範例（Example Scenario）

漏洞場景： 某用戶已登入 shop.com。此時用戶訪問了攻擊者的網站 evil.com。

（1）無 SameSite 防護： evil.com 頁面中包含一個指向 shop.com/api/delete_account 的自動提交表單。由於 Cookie 沒設置 SameSite，瀏覽器發送請求時會自動帶上 shop.com 的 Session Cookie，導致帳號被刪除。

（2）SameSite 屬性分類：

- Strict： 最嚴格。僅允許「同站」請求攜帶 Cookie。從 Google 搜尋結果點進該站，初期也不會帶 Cookie。
- Lax： 權衡型。允許安全的頂級導航（如點擊連結）帶 Cookie，但攔截 POST 等具副作用的跨站請求。
- None： 完全允許跨站傳輸。必須搭配 Secure 屬性（HTTPS），否則現代瀏覽器會拒絕載入。

3. 危險寫法（Vulnerable Code - PHP）

在早期的 PHP 版本或未經優化的設定中，生成 Cookie 時往往忽略了 SameSite 屬性：

```
// index.php
session_start();
// 危險的手動設置方式：僅定義了名稱、值、過期時間與路徑
setcookie("session_id", "xyz789", time()+3600, "/");
// 伺服器回傳的 Header 將呈現：
// Set-Cookie: session_id=xyz789; expires=...; path=/
// 缺少 SameSite 屬性，在舊版瀏覽器中將暴露於 CSRF 風險下
```

4. 安全寫法（Secure Code - PHP）

現代 PHP（7.3+）已支援在 setcookie 函數中以陣列方式配置 SameSite 屬性。

（1）步驟 A：全局 Session 配置

在程式進入點配置，確保 Session ID Cookie 具備 Lax 屬性（目前大多數應用的最佳實踐）：

```
session_start([
    'cookie_lifetime' => 3600,
    'cookie_path' => '/',
    'cookie_secure' => true,   // 必須在 HTTPS 下運作
    'cookie_httponly' => true, // 配合 HttpOnly 防禦 XSS
    'cookie_samesite' => 'Lax' // 關鍵：防禦跨站請求偽造
]);
```

(2) 步驟 B：個別 Cookie 設定

```
$cookie_options = [
    'expires' => time() + 3600,
    'path' => '/',
    'domain' => '',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict' // 針對極度敏感的操作，建議使用 Strict
];
setcookie("auth_token", "secure_value", $cookie_options);
```

5. 開發建議 (Development Recommendations)

從資管與軟體工程實務出發，建議採取以下防禦與管理策略：

(1) 採納「預設 Lax」原則： 雖然 Chrome 80+ 已開始將未標註的 Cookie 視為 Lax，但開發者不應依賴瀏覽器的預設行為，應明確在代碼中宣告以確保不同瀏覽器（如舊版 Safari）的一致性。

(2) 區分業務需求： 對於一般展示性頁面使用 Lax；對於金流、權限變更等高風險模組，應強制使用 Strict。

(3) 處理跨站第三方資源： 若網站功能（如嵌入式小工具）確實需要跨

站傳遞 Cookie，必須設為 SameSite=None 並強制開啟 Secure 屬性。

(4) 監控與審核： 利用 OWASP ZAP 的警報清單，檢查是否有任何 Set-Cookie 指令遺漏了 SameSite 屬性，並將其列入資安合規性掃描的重點項目。

(5) 漸進式升級： 針對舊有系統，若暫時無法全面升級 PHP 版本，可考慮在 Web 伺服器層級（如 Nginx proxy_cookie_path 或 Apache 的標頭重寫規則）統一注入 SameSite 屬性。

（十一） Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)

1. 簡介（Introduction）

(1) X-Powered-By 是一種非標準的 HTTP 回應標頭，通常由 Web 伺服器或後端語言（如 PHP、ASP.NET）自動生成，用以標示目前網頁背後的技術架構。

(2) 其核心問題在於資訊過度揭露（Information Disclosure）。該標頭往往會包含具體的版本號碼（例如 PHP/7.4.3），這無異於向攻擊者提供了一份技術清單。

(3) 攻擊者可以利用這些精確的版本資訊，在 CVE 漏洞資料庫中快速比對該版本是否存在未修補的安全性弱點。這在資安攻擊生命週期中屬於關鍵的「偵察階段（Reconnaissance）」，正確地隱藏此類資訊能有效提升攻擊者的門檻與時間成本。

2. 範例（Example Scenario）

(1) 漏洞偵測： 攻擊者使用簡單的網路工具（如 curl -I 或瀏覽器開發者工具）查看目標網站的響應。

(2) 獲取資訊： 伺服器回傳如下標頭：

HTTP

HTTP/1.1 200 OK

Date: Wed, 25 Mar 2026 10:00:00 GMT

```
Content-Type: text/html; charset=UTF-8
X-Powered-By: PHP/5.6.40
Server: Apache/2.4.6 (CentOS)
```

(3) 發動攻擊： 攻擊者發現該站仍在使用已停止維護（EOL）的 PHP 5.6 版本，隨即搜尋該版本的遠端代碼執行（RCE）漏洞 Payload，並針對性地對網站發動攻擊。

3. 危險寫法 (Vulnerable Configuration)

(1) 在 PHP 的預設配置中，expose_php 選項通常是開啟的，這會自動在每個 HTTP 回應中加入版本資訊。

- php.ini 配置：

```
expose_php = On
```

- 或者在 PHP 代碼中不經意地自定義標頭：

```
<?php
// 此舉會手動暴露後端技術棧
header("X-Powered-By: My-Custom-PHP-Framework-v1.0");
?>
```

4. 安全寫法 (Secure Mitigation)

最根本的解決方案是在系統配置層級將其完全禁用。

(1) 方法 A：修改 php.ini（最推薦）

將 expose_php 設為 Off。修改後需重新啟動 Web 伺服器

```
; php.ini
expose_php = Off
```

(2) 方法 B：在 PHP 腳本中動態移除

若無法修改伺服器設定，可在程式進入點嘗試移除（但效果可能因伺服器環境而異）：

```
<?php
// 嘗試在運行時移除該標頭
header_remove('X-Powered-By');
```

?>

(3) 方法 C：透過 Web 伺服器過濾 (Apache/Nginx)

在 Apache 的 .htaccess 中：

Apache

Header always unset X-Powered-By

在 Nginx 的配置文件中：

Nginx

fastcgi_hide_header X-Powered-By;

proxy_hide_header X-Powered-By;

5. 開發建議 (Development Recommendations)

與系統防禦的角度出發，建議採取以下隱蔽策略：

(1) 循「模糊化 (Obfuscation)」原則： 雖然隱藏資訊並非消除漏洞，但它能落實資安縱深防禦中的「隱晦式安全 (Security through obscurity)」，減少被自動化掃描器鎖定的機率。

(2) 同步禁用 Server 標頭： 除了 PHP 版本，Web 伺服器的版本資訊（如 Server: Apache/2.4.41 (Ubuntu)）也應一併關閉（Apache 設置 ServerTokens Prod 及 ServerSignature Off)。

(3) 定期更新版本： 資訊隱藏只是第一步，核心仍應保持 PHP 版本處於受支援的生命週期（Active Support）內，以確保已修補所有已知漏洞。

(4) 自定義標頭偽裝（進階)： 部分資安專家建議將 X-Powered-By 修改為無關內容（如 X-Powered-By: SuperServer）以誤導攻擊者的判斷，但最穩健的做法仍是直接移除。

(5) 資安稽核： 使用 OWASP ZAP 或 Nessus 等工具進行定期掃描，確認伺服器響應中不再洩漏任何具體的環境架構資訊。

（十二） Server Leaks Version Information via "Server" HTTP Response Header Field

1. 簡介（Introduction）

(1) Server HTTP 標頭 是 Web 伺服器在回傳響應時自動產生的欄位，用於標示處理該請求的伺服器軟體名稱（如 Apache, Nginx, Microsoft-IIS）。

(2) 其核心問題在於環境特徵外洩（Fingerprinting）。預設配置下，伺服器往往會揭露極其詳細的版本號與作業系統資訊（例如 Apache/2.4.41 (Ubuntu) OpenSSL/1.1.1）。

(3) 從資管安全角度看，這種詳盡的技術清單大幅縮短了攻擊者的前期掃描時間。攻擊者只需識別出特定版本，即可針對性地搜尋已知的「一鍵攻擊腳本（One-Click Exploits）」。隱藏版本資訊雖不能根治漏洞，但能有效落實「模糊化防禦」，增加攻擊成本。

2. 範例（Example Scenario）

(1) 漏洞偵察： 攻擊者利用滲透測試工具對目標網站進行 Header 分析。

(2) 獲取標頭： 伺服器在 HTTP 200 OK 的回應中包含以下內容：

HTTP/1.1 200 OK

Server: Apache/2.2.15 (CentOS) DAV/2 PHP/5.4.16

Content-Type: text/html; charset=UTF-8

威脅分析： 攻擊者發現該伺服器使用的 Apache 2.2.15 存在多個已知的高危險漏洞（如 CVE-2017-9798），且 PHP 5.4 已停止更新多年

(3) 發動攻擊：

根據該版本資訊，攻擊者直接加載特定的緩衝區溢位（Buffer Overflow）Payload，成功取得伺服器的控制權（Shell）。

3. 危險寫法（Vulnerable Configuration）

(1) 這是典型的環境配置問題。以最常見的 Apache 與 Nginx 為例，預設設定會揭露完整資訊。

- Apache (httpd.conf):

  # 預設通常為 Full，會顯示版本、OS 與模組資訊

```
ServerTokens Full
ServerSignature On
```

- Nginx (nginx.conf):

```
# 預設為開啟，會在 Header 顯示版本號
server_tokens on;
```

4. 安全寫法 (Secure Mitigation)

必須透過修改 Web 伺服器的核心設定檔來限制標頭輸出的詳細程度。

(1) 方法 A：針對 Apache 伺服器 (httpd.conf / apache2.conf)

應將資訊減至最少，僅顯示為 "Apache"。

```
# 僅顯示軟體名稱，隱藏版本號與作業系統
ServerTokens Prod
# 關閉錯誤頁面底部的伺服器資訊特徵
ServerSignature Off
```

(2) 方法 B：針對 Nginx 伺服器 (nginx.conf)

```
Nginx
http {
    # 隱藏 Nginx 的具體版本號
    server_tokens off;
}
```

(3) 方法 C：透過 PHP 修改 (僅限部分環境)

雖然 Server 標頭通常由網頁伺服器控制，但在某些代理環境下，可嘗試移除：

```
<?php
// PHP 難以直接修改 Server 標頭，應優先修改 Web Server 配置
// 但可確保不手動輸出額外標頭
header_remove("Server");
?>
```

5. 開發建議 (Development Recommendations)

從資管與資訊安全治理的角度出發，建議採取以下多層次防護策略：

(1) 遵循最小資訊揭露原則： 伺服器不應回傳任何對正常用戶無益、但對攻擊者有利的技術細節。將 ServerTokens 設為 Prod 是資安稽核中最基本的要求。

(2) 自定義 Server 標頭 (WAF/Reverse Proxy)： 在進階架構中，可以使用負載平衡器（如 F5）或反向代理（如 Nginx）將 Server 標頭重寫為不具名內容（例如 Server: WebServer)，徹底消除特徵。

(3) 錯誤頁面清理： 確保 404 或 500 錯誤頁面是由應用程式自定義的，避免洩漏伺服器預設的技術簽名。

(4) 自動化配置檢核： 將伺服器配置納入「基礎架構即代碼 (IaC)」的審核流程中，使用指令碼自動檢查 server_tokens 是否已關閉。

(5) 持續更新與補丁管理： 隱藏版本是為了拖延攻擊速度，但根本解決之道仍是定期安裝作業系統與 Web 伺服器的安全更新。

(十三) X-Content-Type-Options Header Missing

1. 簡介 (Introduction)

(1) X-Content-Type-Options 是一個 HTTP 響應標頭，旨在防止瀏覽器執行「MIME 嗅探」。MIME 嗅探是瀏覽器的一種自動機制，當伺服器宣告的 Content-Type 與實際內容不符時，瀏覽器會嘗試猜測並解析該檔案。

(2) 其核心問題在於非預期類型的腳本執行。攻擊者可以上傳一個偽裝成圖片（如 image/jpeg）但實則包含 JavaScript 的惡意檔案。若伺服器未設置 nosniff 指令，瀏覽器可能會忽略圖片標頭，將其作為腳本執行。

(3) 透過設置 X-Content-Type-Options: nosniff，開發者可以強制要求瀏覽器嚴格遵守伺服器所宣告的 MIME 類型。若類型不匹配（例如 Content-Type: text/plain 但內容是腳本)，瀏覽器將直接拒絕執行或載入該資源，進而阻斷潛在的 XSS 攻擊。

2. 範例 (Example Scenario)

(1) 漏洞前提： 某 PHP 網站提供用戶上傳頭像功能，且未配置 nosniff 標頭。

(2) 攻擊路徑：

- 攻擊者創建一個名為 malicious.jpg 的檔案，內容為 <script>alert('XSS')</script>。
- 伺服器接受上傳並將其存放在 /uploads/ 目錄。
- 攻擊者誘導受害者訪問 https://target.com/uploads/malicious.jpg，或者在網頁中以 <script src="/uploads/malicious.jpg"></script> 的方式引用它。

(3) 觸發攻擊： 儘管伺服器回傳 Content-Type: image/jpeg，但瀏覽器在嗅探後發現內容是 JavaScript，進而執行該惡意代碼，導致用戶資料外洩。

3. 危險寫法 (Vulnerable Code - PHP)

(1) 以下 PHP 程式碼僅輸出了內容，卻沒有限制瀏覽器的嗅探行為，這在提供檔案下載或動態生成資源時非常危險：

```
<?php
// download.php
$file_content = file_get_contents("user_upload.txt");
// 即使設置了 Content-Type，若無 nosniff，瀏覽器仍可能進行嗅探
header("Content-Type: text/plain");
echo $file_content;
// 若 $file_content 包含 HTML/JS，瀏覽器可能將其當作網頁解析
?>
```

4. 安全寫法 (Secure Code - PHP)

(1) 正確的做法是在所有響應中加入 nosniff 標記，特別是在處理用戶上傳的資源時。

- 步驟 A：設置 nosniff 標頭

```
<?php
// 強制瀏覽器不進行 MIME 嗅探
header("X-Content-Type-Options: nosniff");
// 確保設置了正確的 MIME 類型
header("Content-Type: text/plain; charset=UTF-8");
echo "這僅僅是一段純文字，瀏覽器不應將其解析為其他格式。";
?>
```

- 步驟 B：全局配置（推薦）

  建議在 Web 伺服器層級統一配置，確保全站生效。

  Apache (.htaccess):

  Apache

  ```
  Header set X-Content-Type-Options "nosniff"
  ```

  Nginx (nginx.conf):

  Nginx

  ```
  add_header X-Content-Type-Options "nosniff" always;
  ```

5. 開發建議 (Development Recommendations)

從資管與資訊安全架構角度出發，建議採取以下多層次防禦策略：

(1) 搭配正確的 MIME 類型： 設置 nosniff 的前提是伺服器必須精確回傳正確的 Content-Type。例如，CSS 檔案必須宣告為 text/css，否則在 nosniff 開啟的情況下，瀏覽器會因安全性考量拒絕加載樣式。

(2) 上傳過濾與重新編碼： 針對上傳功能，應檢查檔案內容是否與副檔名相符。對圖片檔案可進行重新縮放或過濾，移除內嵌的惡意元數據 (Metadata)。

(3) 縱深防禦 (Defense in Depth)： 此標頭應與 Content Security Policy (CSP) 配合使用。即便瀏覽器繞過了 MIME 限制，CSP 也能限制惡意腳本的執行來源。

(4) 禁用不必要的內容類型： 對於 API 響應，建議始終使用 application/json 並加上 nosniff，避免瀏覽器將 JSON 內容誤判為 HTML。

(5) 合規性檢核： 許多資安掃描工具（如 OWASP ZAP）會將缺少此標頭視為「中低風險」漏洞。在資管稽核流程中，應將其列為網頁伺服器基準配置（Baseline Configuration）的必檢查項。

第三節 Web 網站安全評估工具

網站安全評估是確保應用程式防禦能力的關鍵環節。根據測試時機與方法的不同，評估工具主要分為靜態應用程式安全測試（SAST）與動態應用程式安全測試（DAST）。本研究採用的 **OWASP ZAP** 即屬於後者。

一、 動態應用程式安全測試（DAST）概述

(一)DAST 是一種「黑箱測試」技術，在應用程式執行狀態下進行掃描。

(二)其優點在於能模擬駭客的攻擊行為，發現伺服器配置錯誤、認證失效及複雜的邏輯漏洞。

(三)相較於 SAST 僅檢查程式碼語法，DAST 能驗證漏洞在真實環境中的可利用性（Exploitability）。

二、 OWASP ZAP 工具特性分析

OWASP Zed Attack Proxy（ZAP）是目前全球最受歡迎的開源 Web 資安掃描工具，對於資管研究與中小型企業開發流程具備高度價值：

(一)**代理攔截（Intercepting Proxy）：** 允許研究者攔截瀏覽器與伺服器間的 HTTP/HTTPS 流量，進行參數篡改測試。

(二)**自動化掃描（Automated Scan）：** 包含傳統爬蟲（Spider）與 AJAX 爬蟲，能自動探索網站目錄結構並識別輸入點。

(三)**主動與被動掃描：**

被動掃描： 僅分析回應標頭（如 CSP, X-Frame-Options 缺失），不影響

系統運作。

主動掃描： 對目標發送帶有 Payload 的請求，模擬 SQL 注入或 XSS 攻擊。

(四)**擴展性：** 支持 API 整合至 CI/CD 流程，符合 DevSecOps 的現代化開發趨勢。

三、 ZAP 在本研究中的角色

(一)本研究利用 ZAP v2.17 版本針對網頁設計乙級術科題目進行掃描。

(二)透過其產出的風險報告（High, Medium, Low, Informational)，能將抽象的程式碼弱點量化為具體的資安風險，作為後續階段三「程式碼修復」的實證依據。

第四節 PHP 網站因資安事件造成的損失

PHP 作為市佔率極高的後端語言，其安全性直接影響企業的營運連續性。若開發者僅追求功能達成（如乙級技術士考試之要求）而忽略資安防護，將面臨以下層面的重大損失：

一、 資料外洩與個資法規風險

當 PHP 網站存在 SQL Injection 漏洞時，資料庫中的用戶個資、交易紀錄將全數暴露。

（一）**法律成本：** 隨著各國個資法（如台灣《個人資料保護法》、歐盟 GDPR）趨嚴，企業面臨的高額罰鍰常遠超系統開發成本。

（二）**補救成本：** 包含通知受害者、提供信用監測服務及後續的法律訴訟支出。

二、 財務損失與業務中斷

（一）**參數篡改風險：** 如本章第一節所述之 Parameter Tampering，攻擊者可直接修改結帳金額，對電商平台造成直接經濟損失。

（二）**勒索軟體與系統停擺：** 透過檔案上傳漏洞（Unrestricted File Upload）植入後門（Webshell），攻擊者可加密伺服器資料並索要贖金，導致業務中斷。根據統計，企業因資安事件導致的平均停工損失每小時可達數萬美金。

三、 品牌商譽與客戶信任度下降

資安事件具備高度的負面新聞效應。一旦發生點擊劫持（Clickjacking）或 CSRF 攻擊導致用戶帳號遭盜用，客戶對平台的信任感將瞬間瓦解。對於資管領域而言，「信任」是數位轉型的核心，商譽損失往往是無形且難以估計的長期負債。

四、 乙級技術士試題的隱含風險

本研究指出，目前的職訓體系若僅以「功能實作」為合格標準，將產出大量具備功能卻不具防禦力的系統。這些系統若直接上線，將成為資安防護鏈中最脆弱的一環，印證了「不安全的程式碼即是企業未來的負債」之觀點

第三章 研究方法

第一節　實驗環境

本研究的實驗環境可分為五個面向如下

一. 作業系統:

Windows 11 家用版 25H2

二. XAMPP — 本地伺服器環境（Local Server Stack）

（一） 主要功用：

在你的電腦裡模擬出一台網頁伺服器，讓你的 PHP 程式碼動起來的環境。PHP 是一種「伺服器端語言」，它不像 HTML 點兩下就能在瀏覽器開啟，它必須經過伺服器處理。

（二） 包含組件：

Apache： 網頁伺服器負，責接收瀏覽器的請求並回傳網頁。

PHP： 解釋並執行 .php 檔案中的邏輯。

MySQL（MariaDB）： 儲存網站資料的資料庫。

OpenSSL ： 主要為網頁伺服器與資料庫提供加密通訊與身分驗證的功能。

（三） 運作方式：

你必須將程式碼存放在 XAMPP 的 htdocs 資料夾內，並啟動 Apache，才能透過 http://localhost 看到結果。

（四） 版本

軟體名稱	**版本**
XAMPP	8.2.12
Apache	2.4.58
PHP	8.2.12
MariaDB	10.4.32
OpenSSL	3.1.3

三. 程式開發工具

(一) visual studio code

1. 主要功用：是一個功能強大的文字編輯器，但不具備執行 PHP 的能力。
2. 核心特色： 可以安裝各式各樣的擴充功能（例如：Open PHP/HTML/Js Browser）來強化開發體驗。

(二) Open PHP/HTML/Js Browser

1. 主要功用： 讓你在 VS Code 寫完程式後，直接點右鍵就能在瀏覽器中開啟該檔案。
2. 為什麼需要它： 這個套件可以幫你把電腦內部的檔案路徑（如 C:\xampp\htdocs\...）自動轉換成瀏覽器能辨識的網址路徑（http://localhost/...）。

四. OWASP ZAP 弱點掃描工具

(一) 主要功用

1. 攔截代理（Intercepting Proxy）

   ZAP 作為瀏覽器與伺服器之間的「中間人」。

   功能： 您可以攔截請求，並在發送給伺服器之前手動修改參數。

   應用場景： 測試伺服器是否會過濾掉前端無法輸入的非法字元。
2. 爬蟲工具（Spiders）
   (1) 傳統爬蟲（Traditional Spider）： 快速解析 HTML 並提取連結，適合靜態頁面。
   (2) AJAX 爬蟲： 啟動真正的瀏覽器（如 Chrome/Firefox）來執行 JavaScript。
   (3) 關鍵點： 對於您的 PHP 網站，如果使用了大量 JavaScript 進行異步請求，必須合併使用這兩種爬蟲。
3. 被動掃描 vs. 主動掃描

(1) 被動掃描 (Passive Scan)： 僅監控流量，不改變數據。用於發現遺漏的 Security Headers、Cookie 缺少 Secure 屬性等。安全且不會破壞數據。

(2) 主動掃描 (Active Scan)： ZAP 會主動構造攻擊載荷 (Payload)，如發送 ’ OR 1=1 -- 來測試 SQL 注入。

(3) 警告： 僅能對您擁有授權的目標（如您的論文實作網站）執行，因為這可能導致數據損毀。

4. 模糊測試 (Fuzzing)

(1) 功能： 針對特定的輸入欄位，利用預設的字典檔（如 OWASP FuzzDB）發送成千上萬個攻擊變體。

(2) 論文應用： 您可以展示如何手動對 PHP 的 $_POST 參數進行模糊測試，以捕捉程式碼中未預料到的異常。

5. 自動化框架 (Automation Framework)

(1) 這是現代 ZAP 的核心，允許透過一個 YAML 檔案定義整個掃描流程（從環境設定、登入、爬蟲到最後產出報告）

(二) 版本: 2.17.0

(三) 自動掃描 (Automated Scan)

1. 「自動掃描」畫面讓你能夠對應用程式發起自動化掃描——只需在下方輸入 URL 並按下「攻擊 (Attack)」即可。
2. 要攻擊的 URL (URL to attack)，須以 http:// 或 https:// 開頭。 ZAP 必須能夠訪問此 URL 才能進行攻擊。
3. ZAP 會從指定的 URL 開始爬行 (Crawl)，但在開始攻擊前會移除 URL 末尾的斜槓 (/)。這是因為在內部處理中，https://example.com/test/ 會被視為末端節點 (Leaf Node)，這會導致 ZAP 不會去攻擊像 https://example.com/test/1 這樣的 URL。
4. 使用傳統蜘蛛 (Use traditional spider)

(1) 傳統蜘蛛透過尋找 HTML 頁面中的連結來探索應用程式。它的速度非常快，但無法處理大量使用 JavaScript 的應用程式。

(2) 使用 Ajax 蜘蛛 (Use ajax spider) Ajax 蜘蛛透過啟動你選擇的瀏覽器並點擊其發現的連結來探索應用程式。它比傳統蜘蛛慢，但能很好地處理 JavaScript。 此選項僅在安裝了 Ajax 蜘蛛擴充套件 (Add-on) 時才會顯示。如果無法使用，你可以從 ZAP Marketplace 免費下載並安裝。 可用的選項包括：

- 從不 (Never)： 你將需要使用傳統蜘蛛。
- 若是現代應用 (If Modern)： 如果 ZAP 識別出目標為現代化應用程式時會執行——為此你需要配合使用傳統蜘蛛。
- 總是 (Always)： 如果選擇此選項，則無需執行傳統蜘蛛。

五. 靶機

本研究將根據技術士技能檢定網頁設計乙級術科測試應檢參考資料內的四個檢定試題的要求，實作四個網站當作靶機

(一)試題編號 : 17300-102201

名稱 :「卓越科技大學校園資訊系統」網站

網站首頁之完成參考圖:

(二)試題編號 ： 17300-102202

名稱 ：「健康促進網社群平台」網站

網站首頁之完成參考圖:

(三)試題編號: 17300-102203

名稱 ：「ABC 影城」網站

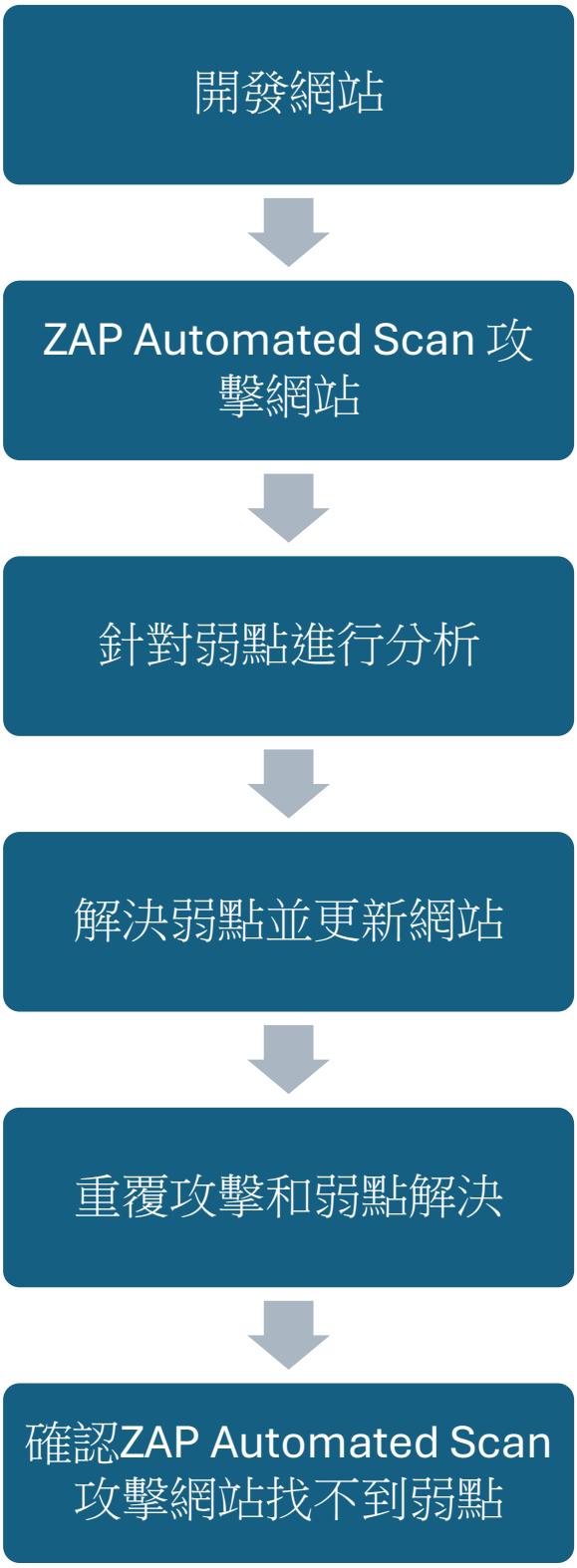
網站首頁之完成參考圖:

(四)試題編號 :17300-102204

名稱 :「精品電子商務」網站

網站首頁之完成參考圖:

第二節　研究流程


開發網站
ZAP Automated Scan 攻擊網站
針對弱點進行分析
解決弱點並更新網站
重覆攻擊和弱點解決
確認ZAP Automated Scan 攻擊網站找不到弱點

第三節　　實驗過程

一.　開發網站

二.　ZAP Automated Scan 攻擊網站

三.　針對弱點進行分析

四.　解決弱點並更新網站

五.　重覆攻擊和弱點解決

六.　確認 ZAP Automated Scan 攻擊網站找不到弱點